

## 1. what is the formula of return of ads

The return on advertising spend (ROAS) is a simple metric that measures the revenue generated for every dollar spent on advertising. It tells you how effective your ad campaigns are.

The formula for ROAS is:

$$\text{ROAS} = \frac{\text{Total Revenue from Ads}}{\text{Total Ad Spend}}$$

For example, if you spend \$1,000 on an ad campaign and it generates \$5,000 in revenue, your ROAS is:

$$\text{ROAS} = \frac{\$5,000}{\$1,000} = 5$$

This means for every dollar you spent, you earned \$5 back in revenue.

## How is ROAS different from ROI?

While ROAS is similar to **return on investment (ROI)**, they are not the same.

- **ROAS** is a **revenue-centric** metric. It only considers ad spend and the revenue directly generated by those ads. It's a key performance indicator (KPI) for marketing and advertising teams.
- **ROI** is a **profit-centric** metric. It takes into account all costs associated with an ad campaign, including production, salaries, and other overheads, to determine the actual profit.

The formula for ROI is:

$$\text{ROI} = \frac{\text{Revenue} - \text{Total Costs}}{\text{Total Costs}} \times 100\%$$

You can have a high ROAS but a low ROI if your total costs are significant. For this reason, a high ROAS is not always a guarantee of profitability.

# —————

what is ads revenue? please use examples to explain

Ad revenue is the income a company generates from displaying advertisements on its platform, website, or app. This revenue model is a primary source of income for many digital businesses, including social media platforms, search engines, news websites, and mobile apps.

There are three main ways ad revenue is calculated and collected:

- **Cost Per Impression (CPM):** The advertiser pays for every **1,000 views** (or impressions) an ad receives. An "impression" is simply one instance of an ad being shown to a user. This model is common for brand awareness campaigns where the goal is to get the ad in front of as many people as possible.
  - **Example:** A brand pays a website a CPM of \$10. If their ad is shown 50,000 times, the website's ad revenue would be:  $(50,000/1,000) \times \$10 = \$500$ .
- **Cost Per Click (CPC):** The advertiser pays the publisher every time a user **clicks** on an ad. This model is popular for direct-response campaigns where the goal is to drive traffic to a specific website or product page.
  - **Example:** A company runs a Google ad campaign with a CPC of \$2. The company's ad is shown 1,000 times and gets 50 clicks. The ad revenue for Google would be:  $50 \times \$2 = \$100$ .

- **Cost Per Action (CPA) / Cost Per Acquisition (CPA):** The advertiser only pays when a user takes a specific, desired **action** after clicking the ad. This action could be a purchase, a form submission, or an app download. This model is very performance-driven and is often used by e-commerce businesses.
  - **Example:** A mobile game company pays an app publisher a CPA of \$5 every time a user downloads and installs their game. If 20 people download the game after seeing the ad, the app publisher's ad revenue is:  $20 \times \$5 = \$100$ .

# — — — — —

what is eCPM, please use example

eCPM, or effective Cost Per Mille, is a metric that measures the revenue generated per 1,000 ad impressions. It's a way for publishers (like websites or apps) to standardize and compare the performance of different ad campaigns, regardless of how they are priced (e.g., CPC, CPA, or CPM).

The formula for eCPM is:

$$\text{eCPM} = \frac{\text{Total Ad Revenue}}{\text{Total Impressions}} \times 1000$$

## Example

Let's say a mobile game developer uses three different ad networks, each with a different pricing model. eCPM allows them to compare which network is most profitable.

| Ad Network | Total Revenue | Impressions | Clicks | Conversions |
|------------|---------------|-------------|--------|-------------|
|------------|---------------|-------------|--------|-------------|

|           |                  |           |       |     |
|-----------|------------------|-----------|-------|-----|
| Network A | \$500 (from CPM) | 1,000,000 | N/A   | N/A |
| Network B | \$300 (from CPC) | 800,000   | 1,500 | N/A |
| Network C | \$150 (from CPA) | 500,000   | 500   | 30  |

匯出到試算表

Now, let's calculate the eCPM for each network:

- **Network A (CPM):**  
eCPM=\$500 / 1,000,000 ×1000 =\$0.50
- **Network B (CPC):**  
eCPM=\$300 / 800,000 ×1000=\$0.375
- **Network C (CPA):**  
eCPM=\$150 / 500,000 ×1000=\$0.30

Based on the eCPM values, **Network A is the most effective** at generating revenue per thousand impressions for this developer, even though it brought in less total revenue than Network B. This insight helps the developer allocate more ad space to Network A to maximize their earnings.

#-----

设计 game ads ranking: optimize for return of ads, follow up optimize for revenue

#-----

Gemini pro

Game ad ranking aims to show users the most relevant ads to maximize revenue for the publisher and effectiveness for the advertiser. The optimization goal determines which ads get shown. You can optimize for **return of ads** (advertiser's perspective) or **revenue** (publisher's perspective).

### 1. Optimize for Return on Ad Spend (ROAS)

Optimizing for ROAS means prioritizing ads that are likely to generate the most profit for the **advertiser**. The goal is to maximize the ratio of revenue to ad spend, ensuring advertisers get a high-quality return. This is typically done by predicting the user's future value (LTV) or conversion probability.

The ranking score can be defined as:

## Ranking Score = Predicted ROAS

- **Prediction Model:** Train a machine learning model to predict a user's future value (e.g., in-app purchases) after seeing an ad.
  - **Features:** Use a combination of user features (demographics, in-game behavior), ad features (genre, creativity), and contextual features (time of day, ad placement).
  - **Target Variable:** The total revenue a user generates for the advertiser over a specific period (e.g., 7 or 30 days).
- **Offline Evaluation:** Evaluate the model's accuracy using metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE) on a holdout dataset.
- **Online Implementation:** In a live environment, the model predicts the ROAS for each potential ad. The ad with the highest predicted ROAS is served to the user. This strategy ensures that advertisers get high-quality users who are likely to become paying customers.

<br> <br>

## 2. Optimize for Revenue (eCPM)

Optimizing for revenue means prioritizing ads that generate the most income for the **publisher** (e.g., the game developer). The goal is to maximize the effective cost per mille (eCPM) of the ad inventory.

The ranking score can be defined as:

### Ranking Score = Predicted eCPM

The predicted eCPM is calculated by multiplying the predicted click-through rate (pCTR) and the predicted conversion rate (pCVR) by the advertiser's bid.

$$\text{Predicted eCPM} = \text{pCTR} \times \text{pCVR} \times \text{Bid} \times 1000$$

Here's how to implement this:

### 1. Prediction Models:

- **pCTR Model:** Train a model to predict the probability that a user will click on a given ad.

- **Features:** User ID, ad ID, ad placement, time of day, ad creative, user's past click history.
  - **Target Variable:** A binary variable (1 for a click, 0 for no click).
- **pCVR Model:** Train a second model to predict the probability that a user will convert (e.g., make a purchase or install an app) after clicking on an ad.
  - **Features:** Similar to the pCTR model but can also include post-click behavior.
  - **Target Variable:** A binary variable (1 for a conversion, 0 for no conversion).
- 2. **Combine the Predictions:** For each potential ad, multiply its predicted pCTR and pCVR with the advertiser's bid to get the predicted eCPM.
- 3. **Ranking:** Rank all potential ads by their predicted eCPM. The ad with the highest predicted eCPM is served to the user.

<br> <br>

## Choosing the Right Strategy

The choice between optimizing for ROAS and eCPM depends on your business goals.

- **ROAS Optimization:** This strategy is beneficial for long-term growth and building a sustainable ad business. By serving high-quality users to advertisers, you build trust and can attract higher-bidding advertisers over time. It's a more sophisticated approach.
- **eCPM Optimization:** This is a more direct and immediate way to maximize revenue. It's often the default choice for many publishers because it's simpler to implement and directly ties to earnings. However, it may sometimes lead to showing lower-quality ads if they have a very high pCTR, even if they don't generate many conversions for the advertiser.

# — — — — — — — — — —

Grok0:

游戏广告排名系统设计

## 1. 设计背景与目标

在游戏场景中，广告排名系统需要平衡广告主的投资回报（ROI, Return on Investment, 即“广告回报”）和平台的整体收入（Revenue）。核心目标是：

首要优化：广告回报（ROI），确保高ROI广告优先展示，提高广告主的满意度和长期合作意愿，避免低效广告浪费资源。

次要优化：平台收入，通过在ROI满足阈值的前提下，优先选择高收入贡献的广告（如高eCPM广告）。

假设游戏广告形式包括插屏广告、激励视频等，数据来源包括历史点击率（CTR）、转化率（CVR）、出价（Bid）等。系统采用实时竞价（RTB）拍卖机制，结合机器学习模型进行排名。

系统假设：

广告库存有限（e.g., 游戏关卡间隙位）。

广告主提供出价（CPC/CPM模式）。

用户侧考虑游戏玩家行为，避免过度打扰影响游戏体验。

## 2. 核心组件

### 2.1 数据输入与特征

广告侧特征：

长度/类型：广告创意时长、格式（视频/静态）。

历史表现：CTR、CVR、ROI历史（ $ROI = (\text{点击}/\text{转化收入} - \text{成本}) / \text{成本}$ ）。

出价：实时Bid（e.g., CPM出价）。

质量分数（QS）：基于广告相关性、加载速度等（0-1分）。

用户/上下文侧特征：

玩家画像：年龄、游戏进度、历史互动。

上下文：游戏场景（关卡结束/加载中）、设备类型。

平台侧：库存价值（eCPM基线）、流量预测。

## 2.2 预测模型

使用机器学习模型（如XGBoost或Deep Learning）预测每个广告的：

预期ROI：  $ROI\_pred = (\text{预计收入} / \text{预计成本})$ 。 预计收入 =  $Bid \times CTR\_pred \times CVR\_pred$ 。

预计成本 = 平台分成比例  $\times$  预计收入。 其中，CTR\_pred 和 CVR\_pred 通过用户-广告匹配模型预测（e.g., Wide&Deep模型）。

预期收入贡献：  $Rev\_pred = eCPM\_pred = Bid \times CTR\_pred \times (1 - \text{分成比例})$ 。

模型训练：

标签：历史ROI和收入数据。

特征工程：嵌入用户行为序列（e.g., 使用Transformer）。

更新频率：实时在线学习，每日离线训练。

## 2.3 排名算法

采用\*\*级联排名（Cascading Ranking）\*\*机制，确保首要优化ROI，次要优化收入：

第一阶段：ROI过滤与排序

阈值过滤：仅保留  $ROI\_pred \geq \theta$ （阈值 $\theta$ 由平台设置，e.g., 1.2，即20%回报率）。低于阈值的广告被淘汰，避免低效投放。

排序：按ROI\_pred降序排名。公式：  $Rank1\_i = ROI\_pred\_i \times QS\_i$ （乘以质量分数，避免垃圾广告）。

第二阶段：收入优化排序



在第一阶段Top-K (e.g., K=10) 候选集中, 按Rev\_pred降序重新排名。公式:  $\text{Rank2}_i = \text{Rev\_pred}_i \times \text{Diversity\_bonus}$  (多样性加成, 避免重复广告主)。

最终排名: 合并两阶段,  $\text{Score\_final} = w1 \times \text{Rank1\_score} + w2 \times \text{Rank2\_score}$  ( $w1 > w2$ , e.g.,  $w1=0.7, w2=0.3$ )。

拍卖机制:

使用第二价格拍卖 (Second-Price Auction): 获胜广告按第二高出价支付。

调整: 如果ROI冲突, 优先高ROI广告, 即使其Rev\_pred稍低。

探索-利用 (Explore-Exploit): 10%流量随机展示新广告, 优化模型。

## 2.4 多样性与公平性

多样性: 使用MMR (Maximal Marginal Relevance) 确保广告类型/主题不重复, 公式:  
 $\text{Score\_div} = \text{Score\_final} - \lambda \times \text{Sim}(\text{当前广告}, \text{已展示广告})$ 。

公平性: 设置曝光上限, 避免单一广告主垄断; 按ROI公平分配预算。

A/B测试: 分流测试不同w1/w2权重, 监控指标。

## 3. 系统架构

前端: 游戏SDK集成, 实时请求广告位。

后端:

广告服务器 (e.g., 使用Kafka实时流)。

排名服务: 微服务, 调用ML模型 (TensorFlow Serving)。

存储: Redis (实时缓存)、Hadoop (历史数据)。

监控指标:

首要: 平均ROI提升率 (目标>15%)。

次要: 总收入增长、Fill Rate (填充率>90%)。

其他: 用户留存率 (避免广告疲劳)。

部署: 容器化 (Docker + K8s), 支持高并发 (游戏峰值QPS>10k)。

#### 4. 潜在挑战与优化

挑战：

数据稀疏：新广告ROI预测不准 → 解决方案：冷启动使用相似广告聚类（K-Means）。

实时性：延迟<100ms → 使用边缘计算。

作弊：虚假点击 → 集成反欺诈模型（e.g., 基于异常检测）。

迭代优化：

短期：基于历史数据调整 $\theta$ 阈值。

长期：引入强化学习（RL），以ROI作为奖励函数动态优化排名。

此设计确保广告回报优先，同时最大化平台收入。如果需要具体代码实现（如Python伪代码）或详细模型公式，请提供更多细节！

#-----

Grok1

#### Game Ads Ranking System Design

I'll outline a system design for ranking advertisements in a game environment, focusing on optimizing for return on ads (interpreting this as maximizing the return from ad impressions, such as user engagement or conversions) as the primary goal, followed by revenue optimization as a secondary objective. This is a classic multi-objective ranking problem, common in ad tech (e.g., similar to Google's Ad Auction or Facebook's ad ranking). The design assumes a mobile/in-app game context where ads (e.g., interstitials, rewarded videos) are shown between game sessions or levels.

The system will use a two-stage ranking pipeline:

**Primary Stage: Optimize for Return** – Rank ads to maximize expected return per impression (e.g., click-through rate (CTR) \* conversion value).

**Secondary Stage: Revenue Boost** – Adjust rankings to incorporate revenue factors (e.g., eCPM) without significantly degrading return.

## Key Assumptions

Ad Inventory: Limited slots per game session (e.g., 3-5 ad placements).

Metrics:

Return: Proxy via expected value (EV) = CTR \* (conversion rate \* reward value). Reward value could be in-game currency or real-world conversions.

Revenue: eCPM (effective cost per mille) or bid price from advertisers.

Data: Historical logs of impressions, clicks, conversions, and revenue per ad creative/campaign.

Constraints: Low latency (<100ms per ranking), privacy compliance (e.g., no user PII), scalability for millions of daily sessions.

Environment: Real-time bidding (RTB) or direct-sold ads.

## High-Level Architecture

The system integrates with the game's ad serving pipeline:

Game Session Trigger (e.g., level complete)



User Context Fetch (device, game progress, past interactions)



Ad Retrieval (from DSPs or internal auction)



[Ranking Engine]

- Stage 1: Return-Optimized Scoring

- Stage 2: Revenue Adjustment



Ad Selection & Serving



Impression Logging → ML Feedback Loop

Components:

Feature Store: Stores real-time user/ad features (e.g., user LTV, ad relevance score).

ML Models: Trained offline, served via online inference (e.g., TensorFlow Serving).

Auction Mechanism: Second-price auction for fairness.

Monitoring: A/B tests for return/revenue trade-offs.

Stage 1: Optimize for Return

Goal: Rank ads to maximize long-term return (e.g., player retention and monetization via engagement). This prevents "spammy" high-revenue but low-engagement ads from dominating.

Scoring Formula:

For each ad  $a$  and user  $u$ :

$$\text{Return Score}(a, u) = p(\text{click}|a, u) \times p(\text{conversion}|a, u, \text{click}) \times V(\text{conversion})$$

$p(\text{click}|a, u)$ : Predicted CTR from a Wide & Deep model.

$p(\text{conversion}|a, u, \text{click})$ : Post-click conversion probability (e.g., install or purchase).

$V(\text{conversion})$ : Expected value (e.g., LTV from conversions).

#### Scoring Formula:

For each ad  $a$  and user  $u$ :

$$\text{Return Score}(a, u) = p(\text{click}|a, u) \times p(\text{conversion}|a, u, \text{click}) \times V(\text{conversion})$$

- $p(\text{click}|a, u)$ : Predicted CTR from a Wide & Deep model.
- $p(\text{conversion}|a, u, \text{click})$ : Post-click conversion probability (e.g., install or purchase).
- $V(\text{conversion})$ : Expected value (e.g., LTV from conversions).

Implementation:

Model: Deep Neural Network (DNN) with embeddings for categorical features (ad category, game genre) and dense features (user session time).

Training: Offline on historical data using logistic loss for CTR, then value-weighted loss for EV.

Exploration: Use epsilon-greedy (5% random) or Thompson Sampling for bandit-style learning.

Ranking: Sort ads by Return Score descending, select top-K for auction.

Why Primary?: Prioritizes game health (e.g., relevant ads keep players engaged longer, increasing future ad opportunities).

Example Pseudocode (Python-like):

```
def compute_return_score(ad, user_features, models):
```

```
    ctr = models['ctr'].predict(ad_features + user_features)
```

```
    conv_rate = models['conv'].predict(ad_features + user_features + [ctr])
```

```
    value = estimate_ltv(ad.category, user_features['game_progress'])
```

```
    return ctr * conv_rate * value
```

```
# In ranking:
```

```
candidate_ads = retrieve_ads(user_context)
```

```
ranked_ads = sorted(candidate_ads, key=lambda a: compute_return_score(a, user_features,  
    models), reverse=True)
```

```
top_ads = ranked_ads[:5]
```

Stage 2: Revenue Optimization

Goal: Fine-tune rankings to boost short-term revenue while preserving >90% of Stage 1 return (enforced via constraints).

**Adjustment Mechanism:** Use a **blended score** with revenue weighting:

$$\text{Final Score}(a, u) = \alpha \times \text{Return Score}(a, u) + (1 - \alpha) \times \text{Revenue Score}(a, u)$$

- $\alpha = 0.7 - 0.9$  (tunable via A/B tests; start high to favor return).
- $\text{Revenue Score}(a, u) = \text{bid\_price}(a) \times p(\text{impression}|a, u)$  (eCPM proxy).

**Alternative: Constrained Optimization:**

- Maximize  $\sum \text{Revenue Score}$  subject to  $\text{Expected Return} \geq \theta \times \text{Stage1 Return}$ .
- Solve via Linear Programming (e.g., PuLP library) or greedy re-ranking.

**Auction Integration:**

- Run a second-price auction on final scores: Winner pays second-highest bid.
- Fallback: If no ads meet return threshold, fall back to pure revenue ranking.

Adjustment Mechanism: Use a blended score with revenue weighting:

$$\text{Final Score}(a, u) = \alpha \times \text{Return Score}(a, u) + (1 - \alpha) \times \text{Revenue Score}(a, u)$$

$\alpha = 0.7-0.9$  (tunable via A/B tests; start high to favor return).

$$\text{Revenue Score}(a, u) = \text{bid\_price}(a) \times p(\text{impression}|a, u) \text{ (eCPM proxy)}.$$

Alternative: Constrained Optimization:

Maximize  $\sum \text{Revenue Score}$  subject to  $\text{Expected Return} \geq \theta \times \text{Stage1 Return}$ .

Solve via Linear Programming (e.g., PuLP library) or greedy re-ranking.

Auction Integration:

Run a second-price auction on final scores: Winner pays second-highest bid.

Fallback: If no ads meet return threshold, fall back to pure revenue ranking.

Implementation:

Model Update: Revenue Score uses a simpler GBDT (Gradient Boosted Decision Tree) for bid prediction.

Re-ranking: Apply blending post-Stage 1, then auction.

Edge Cases: Penalize low-return ads (e.g., score multiplier = 0.5 if return < baseline).

Example Blending:

If Stage 1 ranks: AdA (return=0.8, revenue=0.5), AdB (0.6, 0.9).

With  $\alpha=0.8$ : Final AdA=0.74, AdB=0.66  $\rightarrow$  AdA wins (preserves return).

####-----

请解释这段话

####-----

Grok:

这段话描述了游戏广告排名系统中用于优化广告排序的调整机制，核心目标是平衡收入（Revenue）和玩家回流（Return，代表玩家体验或保留率）。以下是对这段内容的逐部分解释，保持清晰且简洁，同时针对游戏广告排名系统的背景进行说明。

## 1. Adjustment Mechanism（调整机制）

系统使用一个\*\*混合分数（Blended Score）\*\*来综合考虑收入和玩家回流，公式如下：

$$\text{Final Score}(a, u) = \alpha \times \text{Return Score}(a, u) + (1 - \alpha) \times \text{Revenue Score}(a, u)$$

- 解释：
  - $a$ : 广告 (ad) 。
  - $u$ : 用户 (user) 。
  - $\text{Final Score}(a, u)$ : 最终用于排序的广告分数。
  - $\text{Return Score}(a, u)$ : 玩家回流分数，衡量广告对玩家体验的影响（例如，奖励广告可能提升回流率）。
  - $\text{Revenue Score}(a, u)$ : 收入分数，衡量广告的预期收益。
  - $\alpha$ : 权重参数（0.7-0.9，初始偏向回流），通过A/B测试调整。
- 游戏场景：在游戏中，奖励广告（如看视频得金币）通常对回流有正向作用，但收入可能较低；横幅广告可能收入高但降低玩家体验。 $\alpha$  高的设置（如0.8）优先保证玩家不流失。



## 📄 2. Revenue Score (收入分数)

Copy

$$\text{Revenue Score}(a, u) = \text{bid\_price}(a) \times p(\text{impression}|a, u)$$

- **解释:**
  - $\text{bid\_price}(a)$ : 广告主为广告  $a$  的出价。
  - $p(\text{impression}|a, u)$ : 用户  $u$  对广告  $a$  的点击率 (pCTR) 或转化率, 常用作eCPM (每千次展示收益) 的代理。
  - 这是一个标准广告系统收入预测, 类似实时竞价 (RTB) 中的eCPM计算。
- **游戏场景:** 例如, 一个赛车游戏广告可能出价\$0.5, 预测点击率0.05, 则  $\text{Revenue Score} = 0.5 \times 0.05 = 0.025$ 。

## 3. Alternative: Constrained Optimization (替代方案: 约束优化)

除了混合分数, 系统还提供了一个基于约束优化的替代方法:

Maximize  $\sum \text{Revenue Score}$  subject to  $\text{Expected Return} \geq \theta \times \text{Stage1 Return}$

- **解释:**
  - **目标:** 最大化总收入 ( $\sum \text{Revenue Score}$ ) 。
  - **约束:** 保证玩家回流 (Expected Return) 不低于第一阶段回流分数的某个比例 ( $\theta \times \text{Stage1 Return}$ ) 。
  - **实现:** 通过线性规划 (例如Python的PuLP库) 或贪心重排序 (greedy re-ranking) 解决。
- **游戏场景:** 确保即使选择高收入广告, 玩家回流率不会下降超过某阈值 (例如,  $\theta =$

py

- **游戏场景：**确保即使选择高收入广告，玩家回流率不会下降超过某阈值（例如， $\theta = 0.95$  表示回流率不低于第一阶段的95%）。
- **优势：**相比混合分数，约束优化更严格地保证回流，同时优化收入。

#### 4. Auction Integration（拍卖整合）

- **机制：**对最终分数（Final Score）运行**第二价格拍卖（Vickrey Auction）**：
  - 获胜广告支付第二高出价（而非自己的出价），激励真实出价。
- **后备策略：**
  - 如果没有广告满足回流阈值（例如，Return Score < baseline），则回退到**纯收入排名**（只考虑 Revenue Score）。
- **游戏场景：**在游戏暂停界面，如果奖励广告回流分数低，可能切换到高出价的横幅广告以保证收入。

#### 5. Implementation（实现细节）

- **模型更新：**
  - Revenue Score 使用轻量级\*\*GBDT（梯度提升决策树）\*\*预测出价或点击率，计算简单，适合实时场景（延迟<10ms）。
- **重排序：**
  - 第一阶段（Stage 1）生成候选广告后，应用混合分数公式（或约束优化）进行重排序，然后进入拍卖。
- **边缘情况处理：**

💡 Think Harder ×



- **边缘情况处理：**
  - 对低回流广告施加惩罚（如分数乘以0.5），避免展示可能导致玩家流失的广告（例如，侵入式广告）。
- **游戏场景：**在游戏中，惩罚可能包括跳过率高的广告（例如，玩家常关闭的弹窗广告）。

## 6. Example Blending（混合示例）

- **场景：**
  - 第一阶段（Stage 1）排名：
    - 广告A: Return Score = 0.8, Revenue Score = 0.5
    - 广告B: Return Score = 0.6, Revenue Score = 0.9
  - 假设  $\alpha = 0.8$ ：
    - 广告A的最终分数:  $0.8 \times 0.8 + (1 - 0.8) \times 0.5 = 0.64 + 0.1 = 0.74$
    - 广告B的最终分数:  $0.8 \times 0.6 + (1 - 0.8) \times 0.9 = 0.48 + 0.18 = 0.66$
  - **结果：**广告A获胜，因为其综合分数更高（ $0.74 > 0.66$ ），优先考虑回流。
- **游戏场景：**广告A可能是奖励视频（高回流，低收入），广告B是横幅广告（高收入，低回流）。选择A更可能让玩家继续游戏。

## 7. 游戏广告系统的背景意义

- **为何需要混合分数：**游戏广告系统不仅追求短期收入，还要考虑长期玩家留存。过度推送高收入但侵入式广告可能导致

## 7. 游戏广告系统的背景意义

- **为何需要混合分数：**游戏广告系统不仅追求短期收入，还要考虑长期玩家留存。过度推送高收入但侵入式广告可能导致玩家流失，降低LTV（终身价值）。
- **为何用GBDT：**GBDT计算效率高，适合实时预测点击率或出价，尤其在游戏事件触发（如关卡结束）时需要快速响应。
- **为何第二价格拍卖：**确保广告主诚实出价，同时系统能以较低成本获得高价值广告。
- **边缘情况：**游戏中可能有离线模式或新用户（冷启动），需要后备策略（如默认热门广告）。

## 8. 可能的面试扩展点

- **数学推导：**面试官可能要求详细推导约束优化的线性规划公式，或分析  $\alpha$  的敏感性。
- **代码实现：**要求写出混合分数的Python伪码或PuLP线性规划代码。
- **A/B测试：**讨论如何调整  $\alpha$  或  $\theta$ ，例如通过多臂老虎机（Multi-Armed Bandit）动态优化。
- **游戏特定优化：**如何利用游戏事件（如ProGame系统中的关卡完成）进一步提高 Return Score。

如果需要深入某部分（例如，数学公式推导、代码示例或A/B测试设计），请告诉我，我可以进一步展开！

####-----

Gemini

这是一段关于广告排名系统的优化和调整机制的描述，旨在平衡\*\*广告主回报（ROAS）和平台收入（Revenue）\*\*这两个核心目标。

### 调整机制：混合打分

这段文字首先提出了一个\*\*混合打分（Blended Score）\*\*的方案，用于最终的广告排名。它将两个分数进行加权平均：

- **Return Score**：衡量广告能给广告主带来的回报，比如 ROAS（广告支出回报）或 LTV（用户终身价值）预估。
- **Revenue Score**：衡量广告能给平台带来的收入。这里用 eCPM（每千次展示有效成本）作为代理。 $p(\text{impression}|\alpha, u)$  是预测曝光概率。

公式解释:

$$\text{Final Score}(\alpha, u) = \alpha \times \text{Return Score}(\alpha, u) + (1 - \alpha) \times \text{Revenue Score}(\alpha, u)$$

- $\alpha$ ：一个在 0 到 1 之间的权重，用来调节 Return Score 和 Revenue Score 的重要性。
- $\alpha = 0.7-0.9$ ：这是一个经验值，推荐从一个较高的值开始（比如 0.8），表明系统倾向于优先考虑广告主的回报。这个值可以通过 **A/B 测试**进行微调，以找到最佳的平衡点。

## 替代方案：约束优化

除了简单的加权混合，另一种更复杂的方案是约束优化。

- **目标:** 在满足特定回报要求的前提下，最大化平台收入。
- **约束:**  $\text{Expected Return} \geq \theta \times \text{Stage1 Return}$ 。这意味着，最终的广告集合所带来的预期回报必须达到初筛阶段（Stage 1）回报的某个比例  $\theta$ 。
- **求解方法:** 这是一个典型的线性规划问题，可以用数学库（如 PuLP）或贪心算法来解决。

## 竞价整合

这段话描述了将最终的打分集成到拍卖系统中的方式：

- **二次拍卖:** 排名胜出的广告，其出价等于第二名广告的最终打分。这是一种常见的竞价机制，可以鼓励广告主报出其真实的出价。
- **降级机制:** 如果没有广告能满足预先设定的回报阈值（比如预估 ROAS 低于一个基线），系统会放弃复杂的混合打分，退回到纯粹的收入排名，确保平台至少能获得一定的收益。

## 实施细节

- **模型更新:** `Revenue Score` 的计算使用一个更简单的 GBDT (Gradient Boosted Decision Tree) 模型来预测广告的竞价。
- **重排:** 混合打分的逻辑是在精排之后进行的，这意味着它是一个重排 (Re-ranking) 步骤。
- **边缘情况:** 对于那些预估回报极低的广告，可以进行惩罚（比如将它们的最终分数乘以 0.5），以防止它们被意外地选入。

## 示例

举例说明了混合打分机制如何工作。

- 两个广告：`AdA` 的回报高但收入低；`AdB` 的回报低但收入高。
- 当  $\alpha = 0.8$  时， $\text{Final Score}(\text{AdA}) = 0.8 \times 0.8 + 0.2 \times 0.5 = 0.74$ 。
- $\text{Final Score}(\text{AdB}) = 0.8 \times 0.6 + 0.2 \times 0.9 = 0.66$ 。
- 最终 `AdA` 的得分更高，因此它胜出。这个结果表明，该系统倾向于保护广告主的利益 (**Return Score**)。

####-----

## Data Pipeline & ML Training

### Offline Training:

Data: ClickHouse/Spark for logs (features: ad metadata, user embeddings from game telemetry).

Metrics: Optimize for EV (primary), then revenue delta.

Frequency: Daily retraining, weekly A/B tests.

Online Serving:

Latency: Cache user features in Redis; inference via ONNX.

Scale: Kubernetes for horizontal scaling.

Evaluation:

| Metric       | Definition                         | Target                         |
|--------------|------------------------------------|--------------------------------|
| Return Lift  | $\Delta(\text{EV per impression})$ | +10-20% vs. baseline           |
| Revenue Lift | $\Delta(\text{eCPM})$              | +5-15% without >5% return drop |
| Diversity    | Unique ad categories shown         | >80%                           |
| Latency      | End-to-end ranking                 | <50ms                          |

### Potential Challenges & Mitigations

Cold Start: New ads/users → Use content-based similarity (e.g., TF-IDF on ad descriptions) or average scores.

Ad Fatigue: Track frequency caps; decay scores for over-served ads.

Bias: Audit for genre biases (e.g., action games favor violent ads); use fairness constraints.

Regulatory: Comply with GDPR/CCPA; anonymize features.



This design balances player experience (return) with business needs (revenue). For production, prototype with simulated traffic and iterate via A/B tests. If you need code snippets, diagrams, or refinements (e.g., specific ML frameworks), let me know!

# — — — — —

roi is the metric for advertiser, or for ads platform, e.g., google?

In the context of your game ads ranking system, **ROI (Return on Investment)** is typically a metric for the **advertiser**, not the ads platform (e.g., Google or the game platform). Here's a clear breakdown:

- **For Advertisers**:

ROI measures the efficiency of their ad spend. It's calculated as:

**ROI = (Revenue from Ads - Cost of Ads) / Cost of Ads**.

For example, if an advertiser spends \$100 on ads and generates \$150 in revenue (e.g., from in-app purchases or conversions), their ROI is  $(\$150 - \$100) / \$100 = 0.5$  or 50%. Advertisers prioritize high ROI to ensure their campaigns are profitable.

- **For the Ads Platform (e.g., Google, game platform)**:

The platform's primary metric is **revenue** (e.g., total ad revenue or eCPM). While platforms care about advertisers' ROI to maintain long-term relationships (happy advertisers keep spending), their direct goal is to maximize revenue from ad impressions or clicks. Platforms may indirectly optimize for advertiser ROI to ensure ad quality and prevent churn, but their core focus is revenue.

### ### In Your Design Context

Your requirement specifies optimizing for **"return of ads" (ROI)** first, then **revenue**. This implies:

- **Primary optimization (ROI)**: Prioritize ads that deliver high ROI for advertisers, ensuring their campaigns are effective (e.g., high CTR, CVR, or conversions relative to cost). This keeps advertisers satisfied and encourages continued ad spend.

- **\*\*Secondary optimization (revenue)\*\***: Within the pool of high-ROI ads, select those that maximize the platform's revenue (e.g., higher eCPM or bid).

For example:

- An ad with high ROI (good conversions for the advertiser) but low bid might be prioritized over a high-bid ad with poor ROI to ensure advertiser satisfaction.
- Among ads with similar ROI, the platform picks the one with higher revenue potential (e.g., higher CPM).

### ### Clarification for Implementation

If you meant ROI as a platform metric (e.g., platform's profit margin per ad), please confirm, and I can adjust the design to focus on platform ROI (e.g., (platform revenue - operational costs) / costs). Otherwise, the current design assumes ROI for advertisers, with revenue as the platform's secondary goal.

If you need Python code to implement this (e.g., extending the `StringBatcher` class to include ROI-based ranking or a separate ranking module), please specify:

- How ROI is calculated (e.g., data inputs like revenue, cost).
- Example data (e.g., ads with bids, CTR, CVR).
- Whether to integrate with the batching logic (e.g., batch ads by length and rank by ROI).

Let me know your preference or any additional details!

# \_\_\_\_\_

are cpm, cpc, cpa, ecpm metrics for advertiser, or for ads platform, e.g., google?

The metrics **\*\*CPM\*\***, **\*\*CPC\*\***, **\*\*CPA\*\***, and **\*\*eCPM\*\*** are relevant to both advertisers and ad platforms (e.g., Google, game platforms), but their primary use and perspective differ depending on the stakeholder. Below, I clarify each metric and its primary association, focusing on the context of your game ads ranking system.

### ### 1. \*\*CPM (Cost Per Mille/Thousand Impressions)\*\*

- **Definition**: Cost an advertiser pays for 1,000 ad impressions (views), regardless of clicks or conversions.
- **Primary Stakeholder**: **Advertiser**.
- **Advertiser Perspective**: CPM is a cost metric. Advertisers use it to budget campaigns focused on brand awareness, where impressions matter more than clicks or conversions. They aim for lower CPM for cost efficiency.
- **Platform Perspective**: The platform earns revenue per 1,000 impressions based on the CPM bid. Higher CPM bids are preferred as they directly increase platform revenue.
- **In Your System**: CPM is an input for ranking (e.g.,  $\text{Rev\_pred} = \text{CPM} \times \text{CTR\_pred}$  for revenue optimization). Platforms prioritize high CPM ads in the revenue optimization phase, but advertisers focus on CPM relative to their ROI.

### ### 2. \*\*CPC (Cost Per Click)\*\*

- **Definition**: Cost an advertiser pays for each click on their ad.
- **Primary Stakeholder**: **Advertiser**.
- **Advertiser Perspective**: CPC is critical for performance campaigns (e.g., driving website visits). Advertisers aim for lower CPC to maximize clicks within their budget, while ensuring clicks lead to conversions for positive ROI.
- **Platform Perspective**: The platform earns revenue per click based on the CPC bid. Platforms prefer higher CPC bids for revenue but must balance with click likelihood (CTR).
- **In Your System**: CPC contributes to ROI calculation (e.g.,  $\text{Cost} = \text{CPC} \times \text{Predicted Clicks}$ ). Ads with lower CPC but high CTR/CVR may yield better ROI for advertisers, influencing the primary ranking phase.

### ### 3. \*\*CPA (Cost Per Action/Acquisition)\*\*

- **Definition**: Cost an advertiser pays for a specific action (e.g., purchase, sign-up, app install).
- **Primary Stakeholder**: **Advertiser**.

- **Advertiser Perspective**: CPA is the key metric for conversion-focused campaigns. Advertisers target lower CPA to maximize ROI, as it directly ties to their business outcomes (e.g., revenue per install).
- **Platform Perspective**: Platforms offering CPA campaigns earn revenue when actions occur. They use predicted CVR (conversion rate) to estimate CPA revenue but focus less on CPA itself compared to advertisers.
- **In Your System**: CPA is central to ROI optimization (e.g.,  $ROI = (Revenue\_per\_action - CPA) / CPA$ ). Ads with lower CPA and higher conversion value are prioritized in the ROI phase.

#### ### 4. **eCPM (Effective Cost Per Mille)**

- **Definition**: A normalized metric representing revenue per 1,000 impressions, calculated across pricing models (e.g.,  $eCPM = CPC \times CTR \times 1,000$  or  $CPA \times CVR \times CTR \times 1,000$ ).
- **Primary Stakeholder**: **Platform**.
- **Platform Perspective**: eCPM is the primary revenue metric for platforms, as it standardizes revenue across CPM, CPC, and CPA campaigns. Platforms aim to maximize eCPM to optimize total ad revenue.
- **Advertiser Perspective**: Advertisers may monitor eCPM indirectly to compare campaign efficiency across platforms, but it's less critical than CPM, CPC, or CPA, which directly impact their costs.
- **In Your System**: eCPM is key for the secondary revenue optimization phase. After filtering for high-ROI ads, the system ranks ads by eCPM to maximize platform revenue (e.g.,  $Rev\_pred = eCPM\_pred$ ).

#### ### Summary in Context of Your System

- **Advertiser Metrics (Primary)**: CPM, CPC, CPA are costs advertisers optimize to achieve high ROI. Your system's primary goal (optimizing ROI) focuses on these, ensuring ads deliver value to advertisers (e.g., low CPA with high conversions).
- **Platform Metric (Secondary)**: eCPM is the platform's key metric for revenue optimization. Your system uses eCPM in the second ranking phase to prioritize high-revenue ads among those with acceptable ROI.

#### ### Example in Your Game Ads System

- **Input Data**:

- Ad 1: CPM = \$5, CTR\_pred = 2%, ROI\_pred = 1.5.

- Ad 2: CPC = \$0.5, CTR\_pred = 3%, CVR\_pred = 10%, ROI\_pred = 1.8.

- Ad 3: CPA = \$10, CVR\_pred = 5%, ROI\_pred = 1.2.

- **ROI Phase**: Rank by ROI\_pred (Ad 2 > Ad 1 > Ad 3, assuming Ad 3 falls below ROI threshold  $\theta = 1.3$ ).

- **Revenue Phase**: Among Ad 1 and Ad 2, calculate eCPM (e.g., Ad 2 eCPM = CPC × CTR × 1,000 = \$0.5 × 0.03 × 1,000 = \$15). Rank by eCPM if ROI is similar.

# —————

what is the margin ads margin function? is the ads margin for advertiser, or ads platform? please use examples

What is the Ads Margin Function?

In the context of digital advertising (e.g., platforms like Google Ads, Facebook Ads, or Amazon Ads), the "ads margin function" typically refers to margin-based optimization or margin-based bidding/targeting. This is a strategy or algorithmic function used to allocate ad spend, set bids, or prioritize campaigns based on profit margins (the difference between revenue and costs, expressed as a percentage). The goal is to maximize overall profitability rather than just revenue or return on ad spend (ROAS).

**Core Concept:** Traditional ad optimization often focuses on ROAS (revenue per dollar spent on ads) or clicks/impressions. In contrast, a margin function adjusts for product-specific or campaign-specific profit margins by factoring in costs like cost of goods sold (COGS), shipping, platform fees, and overhead. This ensures ads promote high-margin items more aggressively while deprioritizing low-margin ones to avoid unprofitable sales.

**How It Works:** Advertisers or platforms input margin data (e.g., per product or SKU). The function then:

Calculates expected profit on ad spend (POAS) = (Revenue from ads - Ad costs - Other costs) / Ad spend.

Adjusts bids dynamically: Higher bids for high-margin products to capture more traffic, lower bids for low-margin ones.

Optimizes at scale using machine learning (e.g., in Google Ads or Amazon's tools) to balance volume and profitability.

This function is often implemented in bidding algorithms or campaign management tools to automate decisions, reducing manual segmentation by ROAS or margin buckets.

Is the Ads Margin for Advertisers or Ad Platforms?

The ads margin function is primarily for advertisers (brands, e-commerce sellers, or agencies running campaigns), as it helps them optimize their ad budgets for net profit rather than gross revenue. Ad platforms (e.g., Google, Meta, Amazon) provide the tools and infrastructure to implement it but do not "own" the margin—the platforms earn their revenue through fees (e.g., 15-30% take rate on ad auctions) regardless of the advertiser's margins.

For Advertisers: It's a profitability tool to ensure ad spend doesn't erode profits. Platforms like Google Ads require advertisers to upload gross profit margin data per product/category for features like "Gross Profit Optimization" (launched in 2024), which automates bidding to target higher-margin conversions.

For Ad Platforms: Margins aren't directly applicable, as platforms operate on a marketplace model (auction-based revenue). They might use aggregate data to refine algorithms (e.g., favoring high-engagement ads), but the margin function is advertiser-driven. Platforms benefit indirectly by encouraging more efficient spending, which increases overall ad volume.

In agencies (intermediaries between advertisers and platforms), "margin" often means their service fee (e.g., 15-20% of ad spend), but this is distinct from the product/campaign margin function.

## Examples

Here are practical examples to illustrate the ads margin function in action:

E-commerce Seller on Amazon Ads (Advertiser-Focused):

Scenario: An online clothing store sells two T-shirts: Shirt A (costs \$5 to produce/ship, sells for \$20 → 75% margin) and Shirt B (costs \$12 to produce/ship, sells for \$20 → 40% margin). They spend \$1,000/month on ads.

Without Margin Function: Bidding is ROAS-based (e.g., aim for \$4 revenue per \$1 spent). Both shirts get equal bids, leading to \$2,000 revenue but only \$600 profit after costs (low due to Shirt B's thin margins).

With Margin Function: Using tools like Quartile's margin-based optimization, the seller inputs COGS and fees. The algorithm bids 2x higher on Shirt A (\$0.50 CPC) vs. Shirt B (\$0.25 CPC). Result: \$1,500 revenue skewed to Shirt A, yielding \$900 profit (50% uplift). The function ensures ad spend  $\leq$  profit potential per conversion.[quartile.com](https://www.quartile.com)

[tromml.com](https://www.tromml.com)

Google Shopping Campaign (Advertiser vs. Platform Role):

Scenario: A gadget retailer advertises on Google Ads with products: High-margin earbuds (60% margin, \$50 sale price) and low-margin cables (20% margin, \$10 sale price). Ad auction costs are rising.

Without Margin Function: Standard Maximize Conversion Value bidding treats both equally, winning bids on cables (high volume but low profit), resulting in \$10,000 revenue but \$3,000 profit.

With Margin Function: Enable Google's Gross Profit Optimization by uploading margins via product feed. The platform's algorithm prioritizes earbuds in auctions (e.g., adjusts bids +20% for high-margin items based on conversion probability). Result: \$8,000 revenue but \$4,200 profit (40% uplift). The advertiser controls margins; Google handles the bidding automation but takes ~20% auction fee on total spend.[adcore.com](https://adcore.com)

[reddit.com](https://www.reddit.com)

Agency Handling Ads for a Client (Agency Margin Layer):

Scenario: A digital agency runs Facebook Ads for a client spending \$10,000 on a campaign. The agency buys inventory at \$2 CPM but charges the client \$2.35 CPM.

Agency Margin: 15% (\$1,500 fee on \$10,000 spend) for their services — transparent "percent of ad spend" model. This is the agency's profit margin, separate from the client's product margins.

Client's Margin Function: The agency then applies margin-based bidding for the client's products (e.g., prioritizing 50%+ margin items). Client sees \$15,000 revenue and \$6,000 profit after costs; agency earns its cut without affecting client margins. If using CPM model, hidden markups could hurt the client (e.g., agency pockets \$8,750 on \$10,000 by buying cheap inventory).[kineticsequence.com](https://www.kineticsequence.com)

In summary, the margin function empowers advertisers to run profitable campaigns by tying ads to business economics, while platforms enable it as a value-add. For best results, start with accurate COGS data and test via A/B campaigns. If this relates to a specific platform or context (e.g., game ads), provide more details for tailored advice!

#-----

## What is the Ads Margin Function

The ads margin function is a mathematical model used in online advertising to optimize an ad campaign's profitability. It's a key concept for advertisers who want to balance ad spend with the revenue generated by those ads.

At its core, the function is designed to maximize the difference between the **total revenue** from an ad campaign and the **total cost** of that campaign, which is the advertiser's profit or "margin."

## Key Components

The ads margin function combines several variables to calculate profitability.

- **Total Revenue:** This is the money earned from the ad campaign. It's often calculated by multiplying the number of conversions (e.g., sales, sign-ups) by the average revenue per conversion.
- **Total Cost:** This is the amount of money spent on advertising. It's typically calculated by multiplying the number of impressions or clicks by their respective cost.

The goal is to find the optimal bidding strategy, ad placement, or target audience that maximizes the margin function.

## Formula and Example

A simplified ads margin function can be expressed as:

$$\text{Profit Margin} = (\text{Conversions} \times \text{Revenue per Conversion}) - (\text{Impressions} \times \text{Cost per Impression})$$

**Example:** An advertiser runs a campaign with the following metrics:

- Impressions: 1,000
- Cost per Impression (CPM): \$10



- Conversions: 20
- Revenue per Conversion: \$50

Using the formula, the profit margin would be:

- Total Revenue =  $20 \times \$50 = \$1,000$
- Total Cost =  $(1,000 / 1,000) \times \$10 = \$10$
- Profit Margin =  $\$1,000 - \$10 = \$990$

Advanced margin functions incorporate more variables, such as user lifetime value (LTV), to make more sophisticated predictions about long-term profitability.

# —————

is the ads margin for advertiser, or ads platform?

The ads margin is for the **advertiser**. It's a metric that measures the profitability of an advertising campaign from the advertiser's perspective.

## Advertiser vs. Ad Platform

- **For the Advertiser:** The ads margin is the **profit** generated from a campaign. It's calculated as the total revenue a campaign brings in minus the total cost of running that campaign. The goal of the advertiser is to maximize this margin.
- **For the Ad Platform:** The ad platform (e.g., Google, Meta) earns revenue from advertisers. The platform's goal is to maximize its own revenue and eCPM (effective Cost Per Mille), not the advertiser's margin. However, a platform that consistently provides high margins to its advertisers is likely to attract more ad spend and succeed in the long run.

# —————

## 机器学习系统设计面试题

游戏广告排名系统的优化目标是啥？

Gemini

设计游戏广告排名系统时，优化目标不是单一的，而是多个相互关联、有时甚至相互冲突的指标。一个成功的系统需要在这些目标之间找到一个最佳平衡点。

### 主要优化目标

#### 1. 最大化平台收入 (Publisher Revenue)

这是最直接和最核心的目标。广告平台（即游戏开发商）希望通过展示广告赚取尽可能多的钱。

- 核心指标: **eCPM** (effective Cost Per Mille)。eCPM 是每千次广告展示带来的收入。在竞价广告系统中，eCPM 通常由以下公式决定：  
$$\text{eCPM} = \text{bid} \times \text{pCTR} \times \text{pCVR} \times 1000$$
  
这里，**pCTR** 是预测点击率，**pCVR** 是预测转化率（如安装或付费）。CTR 模型和 CVR 模型都旨在准确预测这两个值。最大化 eCPM 意味着系统会优先展示那些广告主出价高，且用户最可能点击和转化的广告。

#### 2. 最大化广告主收益 (Advertiser Profitability)

仅仅最大化平台收入是不够的。如果广告主得不到好的投资回报，他们就会离开。因此，系统也必须帮助广告主获得高质量的用户。

- 核心指标: **ROAS** (Return on Ad Spend)。ROAS 是广告花费的投资回报率，即广告带来的收入与广告花费的比率。  
$$\text{ROAS} = \text{Revenue from Ads} / \text{Ad Spend}$$
  
系统需要识别并推荐那些能为广告主带来高 ROAS 的用户。这通常通过训练 **ROAS** 预估模型来实现，该模型预测一个用户在安装后带来的长期价值 (LTV)。

### 3. 提升用户体验 (User Experience)

如果广告内容过于泛滥、不相关或侵入性强，用户可能会卸载游戏或应用。

- 核心指标: 用户留存率、应用卸载率、广告点击率（作为用户兴趣的信号）。
- 方法: 排名系统需要确保广告的相关性和多样性。例如，通过在排名后引入\*\*重排 (Re-ranking)\*\*策略，避免连续展示过多同类型或重复的广告，从而提升用户满意度。

## 目标之间的权衡

在实际系统中，这三个目标常常相互冲突，需要进行权衡。

- 一个广告可能 eCPM 很高，但带来的用户质量很差，导致广告主的 ROAS 很低。
- 一个广告可能为广告主带来极高的 ROAS，但如果广告内容不受用户欢迎，可能会损害用户体验。

因此，一个先进的广告排名系统会采用多目标学习 (**Multi-objective Learning**)，将所有目标（如 CTR、CVR、LTV、广告相关性等）都融入到一个统一的排名模型中，通过加权或其它方式来找到一个综合的最优解。

#-----

机器学习系统设计面试题

游戏广告排名系统设计

Gemini

游戏广告排名系统设计

设计一个游戏广告排名系统，其核心目标是在每次广告请求时，从海量广告中，为用户筛选并排序出最可能产生高价值行为（如点击、安装、付费）的广告。这不仅能最大化游戏开发商的收入，也能确保广告主获得高质量的用户。

## 1. 系统目标与挑战

- 核心目标: 在低延迟（通常低于100毫秒）内，为用户返回一个最优的广告列表。
- 关键挑战:
  - 高并发: 广告请求量巨大，系统需要处理每秒数万甚至数十万的并发请求。
  - 低延迟: 用户等待时间必须极短，否则会影响用户体验。
  - 数据规模: 需要处理海量的用户行为数据、广告数据和上下文数据。
  - 模型复杂性: 排名模型需要足够复杂，以准确预测用户行为，但同时要满足低延迟要求。

## 2. 核心架构设计

游戏广告排名系统的典型架构是一个多阶段的漏斗，它将广告筛选和排序过程分解为几个步骤，以平衡性能和准确性。

### a. 粗排 (Retrieval / Candidate Generation)

这个阶段的目标是从海量广告库中，快速召回用户可能感兴趣的几百个候选广告。这个阶段对延迟要求极高，但对准确性要求相对较低。

- 方法:
  - 协同过滤: 基于用户历史行为, 找到与当前用户兴趣相似的广告。
  - 基于内容: 根据广告标签、类别等属性, 召回与用户兴趣相符的广告。
  - 热门/流行度: 召回当前热门或高转化的广告, 作为基础候选项。
- 技术栈: 通常使用倒排索引或向量检索 (如 Faiss) 来实现。

## b. 精排 (Ranking)

这个阶段对粗排阶段筛选出的候选广告进行更精确的排序。这里可以使用更复杂的模型和更丰富的特征。

- 特征工程:
  - 用户特征: 年龄、性别、地理位置、游戏行为 (等级、付费历史等)。
  - 广告特征: 广告主、游戏类别、广告创意 (图片、视频、文案)。
  - 上下文特征: 广告位、请求时间、设备类型。
  - 交叉特征: 用户和广告的交互特征, 如用户是否玩过同类游戏。
- 模型选择:
  - 梯度提升树 (如 **XGBoost**, **LightGBM**) : 在处理结构化数据和稀疏特征时表现出色, 训练速度快。
  - 深度学习模型 (如 **Wide & Deep**) : 能够学习更复杂的特征交互, 尤其是在处理大规模、高维特征时效果更好。

## c. 策略与重排 (Re-ranking)

在精排之后, 可以引入一些策略来调整最终的广告列表, 以实现业务目标。

- 多样性: 避免向用户展示太多相似的广告。例如, 如果精排结果全是同一类型的游戏, 可以手动插入一些其他类型的游戏, 以提高用户体验。
- 新广告探索: 在列表中加入一些新广告, 以收集用户反馈, 解决冷启动问题。这通常通过多臂老虎机 (Multi-armed bandit) 等算法来实现。

### 3. 数据流与系统架构

一个完整的游戏广告排名系统是一个数据闭环。

- 数据收集: 用户的每次广告曝光、点击、安装和付费行为都会被记录下来, 并发送到消息队列 (如 **Kafka**)。
- 离线处理: 批处理系统 (如 **Spark**) 从 **Kafka** 消费数据, 进行清洗、特征工程, 并存储到数据仓库 (如 **HDFS**) 中, 用于模型的离线训练。
- 在线服务:
  - 推荐引擎: 接收用户请求, 协调召回和精排服务。
  - 特征存储: 使用低延迟的数据库 (如 **Redis**) 存储实时特征, 以供在线模型快速查询。
  - 模型服务: 部署训练好的模型, 提供低延迟的推理服务 (如使用 **TensorFlow Serving**)。
- 反馈闭环: 用户的行为数据不仅用于离线训练, 也会实时反馈给在线系统, 用于模型的在线更新或实时策略调整。

### 4. 优化方向

- 延迟优化:
  - 使用分布式缓存 (**Redis**) 存储热门广告和用户画像。

- 使用高性能的编程语言（如 Go, C++）实现核心服务。
- 将召回和精排服务部署在用户地理位置相近的数据中心。
- 业务指标优化:
  - **ROAS 预估**: 训练一个预测 ROAS 的模型，帮助广告主优化广告支出。
  - **多目标学习**: 将点击率（CTR）和转化率（CVR）作为联合优化的目标，以平衡短期点击和长期价值。
  - **A/B 测试**: 持续进行 A/B 测试，以评估新模型、新特征和新策略的真实效果。

#-----

## 机器学习系统设计面试题

游戏广告排名系统的优化目标是啥？

Grok:

游戏广告排名系统的优化目标通常是最大化预期收入（**eCPM**），同时平衡玩家体验以维持长期参与度和保留率。具体来说，目标可以分解为以下几个方面：

1. 最大化**eCPM**（有效千次展示成本）：
  - 定义： $eCPM = bid \times pCTR \times user\_value$ （或转化率  $\times$  LTV）。
  - 说明：通过预测点击率（pCTR）和用户终身价值（LTV），选择能带来最高收入的广告。游戏特定场景可能包括奖励广告的转化率（e.g., 看视频得道具）。
  - 量化：目标提升eCPM 20%（如从\$10到\$12）。
2. 提升点击率（**CTR**）：
  - 定义： $CTR = \text{点击数} / \text{展示数}$ 。
  - 说明：通过精准的广告-用户匹配提高点击率（如匹配游戏主题或玩家偏好）。
  - 量化：目标提升CTR 15%（如从5%到5.75%）。
3. 维持玩家体验与保留率：
  - 定义：保留率 = DAU / 安装用户；玩家体验可通过NPS（净推荐值）或跳出率衡量。
  - 说明：避免广告疲劳（如过频展示）或低质量广告，确保玩家不因广告流失。游戏特定：优先奖励广告（如看视频得金币）以提升参与度。

- 量化：保留率下降不超过5%。

#### 4. 低延迟（实时性）：

- 定义：广告排名端到端延迟（如从请求到展示）。
- 说明：游戏环境要求快速响应，尤其在事件触发（如关卡结束）时。
- 量化：P99延迟<100ms。

#### 5. 探索-利用平衡：

- 定义：在已知高收益广告（利用）和测试新广告（探索）间平衡。
- 说明：通过 $\epsilon$ -greedy或Thompson采样，确保广告库存多样性，优化长期收入。

游戏特定优化：

- 利用游戏事件（如ProGame系统）预测“高价值时刻”（e.g., 玩家升级后更可能看奖励广告）。
- 调整广告频率和位置（e.g., 暂停画面 vs. 商店），避免干扰核心游戏循环。

权衡：收入（eCPM）和玩家体验（保留率）可能冲突，需动态调整（如通过多目标优化或质量分数惩罚低体验广告）。如果面试官要求更深入的数学推导或代码实现，我可以进一步展开！

#-----

机器学习系统设计面试题

游戏广告排名系统设计

Grok

### 游戏广告排名系统设计（Machine Learning System Design Interview）

这是一个典型的ML系统设计面试题，焦点是设计一个用于移动或PC游戏的广告排名系统。该系统需要在游戏内实时插入广告（如暂停画面、商店界面、奖励视频），以最大化平台收入（通过点击、转化或安装）同时最小化对玩家体验的干扰。以下是我对这个问题的结构化回答，模拟面试过程：从问题澄清开始，到高层设计、详细组件、挑战与优化。

#### 1. 问题澄清与假设

在面试中，首先澄清范围：



- **目标**：最大化预期收入 ( $eCPM = bid \times pCTR \times user\_value$ )，其中pCTR是点击率预测，user\_value是用户终身价值 (LTV)。游戏特定：优先考虑玩家参与度，避免广告疲劳。
- **规模**：假设每天1亿活跃用户 (DAU)，每个会话10-20个广告机会；广告库存10万+；QPS峰值10k+。
- **约束**：实时性 (<100ms延迟)，支持A/B测试；游戏环境：事件驱动 (e.g., 关卡完成、商店访问)。
- **成功指标**：收入增长20%、点击率提升15%、玩家保留率不降>5%。
- **假设**：游戏类型为休闲/中核 (如赛车或RPG)；广告格式包括横幅、视频、奖励式；支持实时竞价 (RTB)。

如果面试官有特定偏好 (如焦点在模型上)，我会调整。

## #### 2. 高层设计

系统采用**多阶段管道** (inspired by Snapchat和Facebook的广告系统)，包括候选生成、排名和拍卖。整体架构：

...

[游戏事件流 (Kafka)] --> [候选生成] --> [粗排名] --> [精细排名 + 拍卖] --> [广告服务] --> [游戏UI插入]

|                      |                      |

[离线训练]    [实时特征服务] [监控 & A/B]

...

- **输入**：玩家会话 (e.g., 当前关卡、设备)、广告池。

- **输出**：Top-K广告 (K=5-10)，按分数排序。

- \*\*关键创新（游戏特定）\*\*：整合游戏事件（如ProGame系统）预测玩家“购买窗口”，优先奖励广告以提升LTV。

### #### 3. 详细组件设计

#### ##### 3.1 数据收集与处理

- \*\*来源\*\*：

- \*\*玩家数据\*\*：行为日志（点击、观看时长、跳过）、游戏指标（关卡、货币、排名）、 demographics（年龄、设备）。

- \*\*广告数据\*\*：元数据（创意、类别、bid）、历史性能。

- \*\*上下文\*\*：会话时间、游戏状态（e.g., 疲劳分数）。

- \*\*管道\*\*：使用Kafka流式处理实时事件；BigQuery/Hive存储历史数据。每日聚合成特征表（e.g., 14天滑动窗口，如ProGame）。

- \*\*隐私\*\*：匿名化ID，使用差分隐私。

#### ##### 3.2 特征工程

特征是ML的核心。分类如下（总~100维，游戏特定强调行为序列）：

├ 类别 ┃ 示例特征 ┃ 类型 ┃ 游戏相关性 ┃

├-----┃-----┃-----┃-----┃

├ \*\*用户\*\* ┃ LTV估计、历史CTR、会话深度、偏好embedding（e.g., 动作 vs. 策略游戏） ┃ 数值/嵌入 ┃ 基于游戏进度预测“高价值时刻”（e.g., 升级后） ┃

├ \*\*广告\*\* ┃ 类别one-hot、创意新鲜度、历史转化率 ┃ 分类/数值 ┃ 匹配游戏主题（e.g., 赛车游戏推汽车广告） ┃

|\*\*上下文\*\*|时间/位置、疲劳分数（最近广告暴露）、设备类型|数值/分类|游戏暂停时优先非侵入式|

|\*\*交叉\*\*|用户-广告相似度（cosine sim of embeddings）|嵌入|序列特征：过去14天事件sum/avg（如观看广告数）|

3.2 特征工程

特征是ML的核心。分类如下（总~100维，游戏特定强调行为序列）：

| 类别  | 示例特征                                            | 类型    | 游戏相关性                       |
|-----|-------------------------------------------------|-------|-----------------------------|
| 用户  | LTV估计、历史CTR、会话深度、偏好embedding（e.g., 动作 vs. 策略游戏） | 数值/嵌入 | 基于游戏进度预测“高价值时刻”（e.g., 升级后）  |
| 广告  | 类别one-hot、创意新鲜度、历史转化率                           | 分类/数值 | 匹配游戏主题（e.g., 赛车游戏推汽车广告）     |
| 上下文 | 时间/位置、疲劳分数（最近广告暴露）、设备类型                         | 数值/分类 | 游戏暂停时优先非侵入式                 |
| 交叉  | 用户-广告相似度（cosine sim of embeddings）              | 嵌入    | 序列特征：过去14天事件sum/avg（如观看广告数） |

• 工程：使用Spark预计算离线特征；实时特征从Redis服务。游戏特定：RFM（Recency/Frequency/Monetary）+事件聚合（如ProGame的60特征）。

- \*\*工程\*\*：使用Spark预计算离线特征；实时特征从Redis服务。游戏特定：RFM（Recency/Frequency/Monetary）+事件聚合（如ProGame的60特征）。

##### 3.3 模型设计

多阶段以平衡准确性和速度：

- \*\*候选生成（召回，~1ms）\*\*：Two-tower DNN（TorchRec风格）生成用户/广告embeddings，ANN搜索Top-1000候选（FAISS索引）。训练：对比学习（InfoNCE loss）。

- **粗排名 (~10ms)**: 轻量GBDT (XGBoost) 预测pCTR。标签: 二元点击。特征: ~50维低维。
- **精细排名 (~50ms)**: 深度模型 (Wide&Deep或Transformer) 预测预期价值 =  $\text{bid} \times \text{p}(\text{action}) \times \text{LTV}$ 。
- **p(action)**: 多任务学习 (CTR + 转化率), 使用MLP (单隐藏层100神经元, 如ProGame)。
- 游戏特定: 整合序列模型 (LSTM) 捕捉会话动态 (e.g., 连续失败后推安慰广告)。
- **训练**:
  - 数据: 历史日志 (采样负例平衡)。
  - 损失: 加权BCE (正样本权重高); 探索-利用 ( $\epsilon$ -greedy) 注入多样性。
  - 框架: PyTorch, 分布式训练 (Ray Train)。
- **更新**: 每日增量训练; 冷启动: 内容-based过滤。

#### #### 3.4 排名与拍卖

- **分数计算**:  $\text{eCPM} = \text{bid} \times \text{pCTR} \times \text{quality\_score}$  (用户满意度代理)。
- **机制**: 第二价格拍卖 (Vickrey), 加ML调整 (e.g., 惩罚低质量广告)。
- **游戏优化**: 位置拍卖 (e.g., 商店位 vs. 暂停位), 基于上下文权重。

#### #### 3.5 评估与监控

- **离线**: NDCG@K (排名质量)、AUC (CTR预测); 模拟拍卖收入。
- **在线**: A/B测试 (e.g., 新模型 vs. baseline), 指标: eCPM、保留率、ROAS。使用MLflow跟踪。
- **监控**: Prometheus警报漂移; 游戏特定: 玩家反馈循环 (NPS分数)。

| 指标   | 目标     | 计算方式                                            |
|------|--------|-------------------------------------------------|
|      |        |                                                 |
| eCPM | +20%   | $\sum(\text{bid} \times \text{点击}) / \text{印象}$ |
| CTR  | +15%   | 点击 / 印象                                         |
| 保留率  | 不降>5%  | DAU/安装                                          |
| 延迟   | <100ms | P99端到端                                          |

3.5 评估与监控

- 离线：NDCG@K（排名质量）、AUC（CTR预测）；模拟拍卖收入。
- 在线：A/B测试（e.g., 新模型 vs. baseline），指标：eCPM、保留率、ROAS。使用MLflow跟踪。
- 监控：Prometheus警报漂移；游戏特定：玩家反馈循环（NPS分数）。

| 指标   | 目标     | 计算方式                                            |
|------|--------|-------------------------------------------------|
| eCPM | +20%   | $\sum(\text{bid} \times \text{点击}) / \text{印象}$ |
| CTR  | +15%   | 点击 / 印象                                         |
| 保留率  | 不降>5%  | DAU/安装                                          |
| 延迟   | <100ms | P99端到端                                          |

4. 规模化与挑战

- 规模：分片服务（Kubernetes），缓存热门广告；CDN分发创意。
- 挑战 & 解决方案：

#### 4. 规模化与挑战

- \*\*规模\*\*：分片服务（Kubernetes），缓存热门广告；CDN分发创意。

- **挑战 & 解决方案**:
  - **实时性**: 异步特征计算; 模型量化 (TensorRT)。
  - **冷启动**: 新玩家用流行广告; 新广告用元数据相似。
  - **探索-利用**: Thompson采样平衡新旧广告。
  - **游戏疲劳**: 上下文过滤 (e.g., 每5min限1广告); 多臂老虎机优化频率。
  - **偏差**: A/B校准; 多样性约束 (e.g., 类别平衡)。
  - **边缘案例**: 离线模式 (无网时默认广告)。

#### #### 5. 后续讨论

- **扩展**: 多模态 (整合游戏截图CV); 联邦学习隐私。
- **面试提示**: 画图解释管道; 量化假设 (e.g., 存储: TB级日志); 问面试官反馈。

这个设计借鉴了Snapchat的多阶段排名、Facebook的推荐引擎和ProGame的游戏事件预测, 适应游戏场景以提升LTV。如果需要代码草图或深入某个部分 (如模型伪码), 我可以扩展!

#-----  
#-----  
#-----  
#-----  
#-----  
#-----  
#-----  
#-----  
#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

# \_\_\_\_\_

# \_\_\_\_\_

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----

#-----